

OBJECT ORIENTED SYSTEMS (OOS) — 2022

QUESTION 3 COMPLETE SOLUTION

Introduction

Welcome to the complete detailed solution of Question 3 (CO2) from the 2022 Object Oriented Systems examination  .

This question focuses heavily on:

-  Exception-prone input handling
-  File handling
-  Multithreading
-  Synchronization
-  Thread control methods
-  Java I/O

These topics are among the most important practical areas of Java programming because they directly relate to real-world application development. The explanations below are based on the uploaded semester question paper and advanced Java concepts for maximum clarity and exam-oriented understanding.

QUESTION 3(a)

 **Consider the following code snippet. Point out the problems and state how to solve them.**

```
Scanner sc = new Scanner(System.in);
int a = sc.nextInt();
System.out.println("a = " + a);
String s = sc.nextLine();
System.out.println("s = " + s);
char ch = sc.nextChar();
System.out.println("ch = " + ch);
```

Answer

This program contains multiple important problems related to:

- Input buffering
- Scanner limitations
- Character input handling

These are extremely common mistakes made by beginners .

Problem 1 — `nextLine()` Skipping Input

After:

```
int a = sc.nextInt();
```

the **ENTER** key remains inside the input buffer.

Thus when:

```
String s = sc.nextLine();
```

executes, it consumes the leftover newline character immediately instead of taking actual string input.

 **Why Does This Happen?** Methods like `nextInt()`, `nextFloat()`, and `nextDouble()` consume only the numeric value. They do NOT consume the **ENTER** key. Thus, a buffer issue occurs.

🌟 Problem 2 — `nextChar()` Does Not Exist

The `Scanner` class has NO method named:

```
nextChar()
```

Thus, a compiler error occurs .

✅ Correct Way to Take Character Input

Use:

```
char ch = sc.next().charAt(0);
```

Explanation:

- `next()` → takes string token
- `charAt(0)` → extracts first character

✅ Corrected Program

```
import java.util.*;

class ScannerDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int a = sc.nextInt();
        sc.nextLine(); // Clears the buffer
        System.out.println("a = " + a);
    }
}
```

```
String s = sc.nextLine();
System.out.println("s = " + s);

char ch = sc.next().charAt(0);
System.out.println("ch = " + ch);
    }
}
```

Sample Input

```
10
Java
A
```

Output

```
a = 10
s = Java
ch = A
```

Important Scanner Methods

Method	Purpose
<code>nextInt()</code>	Integer input
<code>nextFloat()</code>	Float input
<code>nextLine()</code>	Entire line input
<code>next()</code>	Single word input

Conclusion

This question demonstrates common Scanner-related issues involving buffer handling and character input. Proper understanding of input methods is essential for writing robust Java programs 🚀.

QUESTION 3(b)

? **Write a Java class having a method that opens a text file in read mode, selects all words containing at most one vowel, and writes them into another file.**

? **Take source and destination file names from command line arguments.**

Answer

This question combines:

- 📁 File handling
- 💬 String processing
- 🚀 Command line arguments
- 💡 Character analysis

🌟 Problem Understanding

We must:

1. ✅ Read words from source file
2. ✅ Count vowels in each word
3. ✅ Select words having at most one vowel
4. ✅ Write them into destination file

🌟 What Are Vowels?

Vowels are: **a e i o u**. Both uppercase and lowercase must be considered.

✅ Complete Java Program

```
import java.io.*;
import java.util.*;

class FileWordFilter {

    static int countVowels(String word) {
        int count = 0;
        word = word.toLowerCase();

        for(int i = 0; i < word.length(); i++) {
            char ch = word.charAt(i);
            if(ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
                count++;
            }
        }
        return count;
    }

    public static void main(String[] args) throws Exception {
        if(args.length < 2) {
            System.out.println("Provide source and destination files");
            return;
        }

        FileReader fr = new FileReader(args[0]);
        BufferedReader br = new BufferedReader(fr);

        FileWriter fw = new FileWriter(args[1]);
        PrintWriter pw = new PrintWriter(fw);

        String line;
```

```
while((line = br.readLine()) != null) {
    String words[] = line.split("\\s+");

    for(String word : words) {
        if(countVowels(word) <= 1) {
            pw.println(word);
        }
    }

    br.close();
    pw.close();

    System.out.println("Filtered words written successfully");
}
```

☀ Example

Suppose the source file contains:

```
sky
apple
try
book
gym
```

Words having at most one vowel:

```
sky
try
gym
```

☀ Important File Classes Used

Class	Purpose
<code>FileReader</code>	Reads characters from file
<code>BufferedReader</code>	Efficient reading (reads lines)
<code>FileWriter</code>	Writes into file
<code>PrintWriter</code>	Convenient text writing

☀ Role of Command Line Arguments

- `args[0]` → source file
- `args[1]` → destination file

Conclusion

This program demonstrates efficient file processing, string manipulation, and command-line handling in Java



QUESTION 3(c)

? **What is the drawback of creating a thread by extending Thread class?**

? **How can this drawback be overcome?**

? **Show how thread execution can be started.**

Answer

Java provides two ways to create threads:

1. Extending `Thread` class
2. Implementing `Runnable` interface

Drawback of Extending Thread Class

Java does NOT support multiple inheritance. If a class extends `Thread`, then it cannot extend any other class . Thus, flexibility decreases.

Example:

```
class MyThread extends Thread {  
    // Cannot extend any other class now  
}
```

Solution — Implement Runnable Interface

The `Runnable` interface solves this problem because the class remains free to extend another useful class.

Example Using Runnable:

```
class MyTask implements Runnable {  
    public void run() {  
        System.out.println("Thread Running");  
    }  
}  
  
class Demo {  
    public static void main(String[] args) {  
        MyTask task = new MyTask();  
        Thread t = new Thread(task);  
    }  
}
```

```
t.start();  
}  
}
```

Output:

Thread Running

🌟 How Thread Execution Starts

```
t.start();
```

creates a new thread internally. Then the JVM automatically invokes:

```
run()
```

⚠️ Important Difference

Method	Meaning
<code>run()</code>	Normal method call (does NOT start a new thread)
<code>start()</code>	Creates a new thread and then calls <code>run()</code>

🎯 Conclusion

Implementing `Runnable` is preferred over extending `Thread` because it supports better flexibility and object-oriented design 🚀.

📄 QUESTION 3(d)

? **Discuss when synchronized block is better to use than synchronized method.**

✅ Answer

Synchronization is used to prevent: ⚠️ **Race Condition** when multiple threads access shared resources simultaneously.

Java provides:

1. `synchronized` method
2. `synchronized` block

🌟 Synchronized Method

The entire method becomes locked.

```
synchronized void show() {  
    // Critical and Non-critical code  
}
```

☀ Synchronized Block

Only specific critical sections become locked.

```
synchronized(this) {  
    // Only Critical code  
}
```

☀ When Synchronized Block is Better

A synchronized block is preferred when:

- ☒ Only a small portion of the code needs synchronization
- ☒ Better performance is required
- ☒ Entire method locking is unnecessary

💡 **Why Better Performance?** A smaller locked region means less waiting time for other threads, better concurrency, and higher efficiency.

☒ Example

```
class Counter {  
    int count = 0;  
  
    void increment() {  
        System.out.println("Non-critical work"); // Executed concurrently  
  
        synchronized(this) {  
            count++; // Locked critical section  
        }  
    }  
}
```

☀ Benefit

Only `count++` is synchronized. The remaining code executes concurrently. Problem with Synchronized Method: `synchronized void increment()` locks the WHOLE method unnecessarily.

🎯 Conclusion

Synchronized blocks provide finer control and better performance compared to synchronized methods because only critical sections are locked 🚀.

📄 QUESTION 3(e)

? **State functionalities of:**

- `sleep()`
- `join()`
- `wait()`
- `notify()`

? **Provide overloaded versions if any.**

✅ **Answer**

These methods are extremely important for thread coordination and communication 🕒.

☀️ `sleep()`

✅ **Purpose** Temporarily pauses the execution of the current thread.

✅ **Syntax**

```
Thread.sleep(1000); // Pauses for 1000 milliseconds
```

✅ **Overloaded Versions**

```
sleep(long millis)
sleep(long millis, int nanos)
```

✅ **Example**

```
class Demo {
    public static void main(String[] args) throws Exception {
        System.out.println("Start");
        Thread.sleep(2000);
        System.out.println("End");
    }
}
```

☀️ `join()`

✅ **Purpose** Makes one thread wait until another thread finishes execution.

✅ **Overloaded Versions**

```
join()
join(long millis)
join(long millis, int nanos)
```

✓ Example

```
class MyThread extends Thread {
    public void run() {
        for(int i = 1; i <= 5; i++) {
            System.out.println(i);
        }
    }
}

class Demo {
    public static void main(String[] args) throws Exception {
        MyThread t = new MyThread();
        t.start();
        t.join(); // Main thread waits until t finishes
        System.out.println("Main Ends");
    }
}
```

☀ wait()

✓ **Purpose** Makes the current thread wait until notified. Must be used inside a synchronized block/method.

✓ Overloaded Versions

```
wait()
wait(long millis)
wait(long millis, int nanos)
```

☀ notify()

✓ **Purpose** Wakes one waiting thread.

✓ Method Version

```
notify()
```

☀ Difference Between `sleep()` and `wait()`

Feature	<code>sleep()</code>	<code>wait()</code>
Belongs To	<code>Thread</code> class	<code>Object</code> class
Lock Released?	No	Yes
Synchronization Required?	No	Yes

 Conclusion

Thread coordination methods like `sleep()`, `join()`, `wait()`, and `notify()` are essential for efficient multithreaded programming and inter-thread communication 🚀.